GRAPHS IN C++

The Complete Implementation Handbook

12 Questions · Templates · Patterns · Working Code

This doc has two parts:

PART 1 — The Templates (Read this first!)
Core BFS template · Core DFS template · Graph input template
Grid template · When to use what (decision table)

PART 2 — All 12 Questions (Use templates + solve)
Q1 Graph Stats Q2 BFS Traversal Q3 DFS Components Q4 Islands
Q5 Grid BFS Q6 Cycle+Bipartite Q7 Implicit BFS Q8 Knight Moves
Q9 Directed BFS Q10 Directed Cycle Q11 Topo Sort Q12 Build Levels

PART 1: THE TEMPLATES

Learn these patterns. Every graph question is just a variation of these.

Decision Guide: BFS or DFS?

First question you ask when you see a graph problem:

What you need to find	Algorithm	Why
Shortest path / fewest moves / minimum hops	BFS	BFS visits level-by-level, nearest first
Just visit all nodes / count components	DFS	DFS is simpler to write with recursion
Detect a cycle (undirected graph)	DFS + parent	Track parent, flag if non-parent visited
Detect a cycle (directed graph)	DFS + 3 colors	Gray = in-stack, reaching gray = cycle
Topological order (task scheduling)	Kahn's BFS	Process 0-in-degree nodes first
Longest path in DAG	Topo DP	Process in topo order, update dp[v]
Flood fill / island counting	DFS or BFS	Either works, DFS shorter to write
Check bipartite (2-colorable)	BFS 2-coloring	Color alternately, check conflict

Template 0 — Reading a Graph (Always Start Here)

Copy this every time. Change only undirected/directed as needed.

Undirected Graph (most common)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int N, M;
    cin >> N >> M;    // N = nodes, M = edges

    vector<vector<int>> adj(N);  // adj[i] = list of neighbors

    for (int i = 0; i < M; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);   // BOTH directions for undirected
        adj[v].push_back(u);   // <--- never forget this line
    }

    // Optional: sort neighbors (when question needs ascending order)
    for (int i = 0; i < N; i++) sort(adj[i].begin(), adj[i].end());

    // ... your BFS or DFS here
}
```

Directed Graph

```
    // For directed graph: only ONE direction
    adj[u].push_back(v);   // u -> v only, NOT v -> u

    // Also track in-degree if needed (for topo sort)
    vector<int> in_degree(N, 0);
    in_degree[v]++;
```

> ⚠ **CRITICAL RULE**
> GOLDEN RULE: Undirected = add BOTH ways. Directed = add ONE way.
> Forgetting this is the #1 bug that kills marks in exams.

Template 1 — BFS (Breadth First Search)

BFS in one line

USE BFS WHEN: shortest path, minimum moves, fewest hops, level-by-level exploration.

KEY TOOL: queue<int> → push to back, pop from front (FIFO).

HOW IT WORKS: Explore all nodes at distance 1 first, then distance 2, etc.

```
// ═══════════════════════════════════
// CORE BFS TEMPLATE — MEMORIZE THIS
// ═══════════════════════════════════

vector<int> dist(N, -1);   // -1 means 'not visited'
queue<int> q;

// STEP 1: Start node
dist[S] = 0;
q.push(S);

// STEP 2: Process until queue empty
while (!q.empty()) {
    int u = q.front();
    q.pop();

    // STEP 3: Visit each neighbor
    for (int v : adj[u]) {
        if (dist[v] == -1) {      // not visited yet
            dist[v] = dist[u] + 1;  // one more step
            q.push(v);
        }
    }
}

// After BFS:
// dist[node] = shortest distance from S to node
// dist[node] == -1 means unreachable
```

BFS on a Grid (for Q5, Q8, Q7-style problems)

```
// Grid BFS — swap adj[u] loop with 4-direction neighbors
int dr[] = {-1, +1,  0,  0};   // up, down, left, right
int dc[] = { 0,  0, -1, +1};

vector<vector<int>> dist(R, vector<int>(C_size, -1));
queue<pair<int,int>> q;
dist[sr][sc] = 0;
q.push({sr, sc});

while (!q.empty()) {
    auto [r, c] = q.front();
    q.pop();

    for (int d = 0; d < 4; d++) {
        int nr = r + dr[d];
        int nc = c + dc[d];
        // Valid = inside grid, not a wall, not visited
        if (nr >= 0 && nr < R && nc >= 0 && nc < C_size
            && grid[nr][nc] == 0 && dist[nr][nc] == -1) {
```

```
            dist[nr][nc] = dist[r][c] + 1;
            q.push({nr, nc});
        }
    }
}
```

Template 2 — DFS (Depth First Search)

DFS in one line

USE DFS WHEN: counting components, flood fill, cycle detection, topological sort.
KEY TOOL: recursion (function calls itself). The call stack IS the DFS stack.
HOW IT WORKS: Go as deep as possible, then backtrack.

```
// ═══════════════════════════════════
// CORE DFS TEMPLATE — MEMORIZE THIS
// ═══════════════════════════════════

vector<bool> visited(N, false);

void dfs(int u, vector<vector<int>>& adj) {
    visited[u] = true;   // mark BEFORE exploring neighbors

    for (int v : adj[u]) {
        if (!visited[v]) {
            dfs(v, adj);   // go deeper
        }
    }
    // All neighbors done. u is fully explored.
}

// To explore whole graph (even disconnected parts):
for (int i = 0; i < N; i++) {
    if (!visited[i]) {
        dfs(i, adj);
        // Every time we enter here = new component!
    }
}
```

DFS with 3 Colors (Directed Cycle Detection)

```
// 0 = WHITE (not visited), 1 = GRAY (in progress), 2 = BLACK (done)
vector<int> color(N, 0);
bool has_cycle = false;

void dfs3(int u, vector<vector<int>>& adj) {
    color[u] = 1;   // GRAY: now exploring

    for (int v : adj[u]) {
        if (color[v] == 1) has_cycle = true;  // GRAY = cycle!
        if (color[v] == 0) dfs3(v, adj);      // WHITE = explore
        // BLACK = done, safe to skip
    }
    color[u] = 2;   // BLACK: fully done
}
```

DFS on Grid (Flood Fill)

```
int dr[] = {-1, +1,  0,  0};
int dc[] = { 0,  0, -1, +1};

void dfs_grid(int r, int c) {
    visited[r][c] = true;
```

```
    for (int d = 0; d < 4; d++) {
        int nr = r + dr[d], nc = c + dc[d];
        if (nr >= 0 && nr < R && nc >= 0 && nc < C
            && grid[nr][nc] == 1 && !visited[nr][nc]) {
            dfs_grid(nr, nc);
        }
    }
}
```

Template 3 — Kahn's Algorithm (Topological Sort)

Kahn's in one line

USE WHEN: ordering tasks with prerequisites, detecting cycles in directed graphs.
IDEA: Repeatedly pick the node with no remaining prerequisites (in-degree = 0).
CYCLE DETECTION FREE: if output count < N, there's a cycle.

```
vector<int> in_degree(N, 0);
// Fill in_degree while reading edges: in_degree[v]++ for each u->v

// Use min-heap for lex-smallest order (use regular queue if order doesn't matter)
priority_queue<int, vector<int>, greater<int>> pq;
for (int i = 0; i < N; i++)
    if (in_degree[i] == 0) pq.push(i);

vector<int> topo;
while (!pq.empty()) {
    int u = pq.top(); pq.pop();
    topo.push_back(u);

    for (int v : adj[u]) {
        in_degree[v]--;
        if (in_degree[v] == 0) pq.push(v);
    }
}

if (topo.size() < N) // CYCLE detected
else                 // topo[] is the topological order
```

PART 2: ALL 12 QUESTIONS

Each question: What it asks · Brain logic · Full code · Example trace

Q1 Graph Stats — Adjacency Lists & Degrees

Difficulty: ★★ Intermediate

What it Asks

Read an undirected graph. Print: total vertices, total edges, degree of each vertex, and max degree.

DEGREE of a node = number of edges connected to it = adj[i].size()

Brain Logic (before touching code)

1. Read N and M.
2. For each edge (u,v): add v to adj[u] AND u to adj[v] (undirected = both ways).
3. Degree of vertex i = adj[i].size() — that's it, no extra work!
4. Loop all vertices, track maximum degree.
5. Print.

Full Code

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int N, M;
    cin >> N >> M;

    vector<vector<int>> adj(N);

    for (int i = 0; i < M; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);   // undirected: both ways
    }

    int maxDeg = 0;
    for (int i = 0; i < N; i++)
        maxDeg = max(maxDeg, (int)adj[i].size());

    cout << "Vertices: " << N << "\n";
    cout << "Edges: " << M << "\n";

    cout << "Degrees:";
    for (int i = 0; i < N; i++) cout << " " << adj[i].size();
    cout << "\n";

    cout << "Max Degree: " << maxDeg << "\n";
    return 0;
}
```

Example

INPUT	OUTPUT
5 5 0 1 0 2 1 2 1 3 3 4	Vertices: 5 Edges: 5 Degrees: 2 3 2 2 1 Max Degree: 3

Q2 BFS Traversal & Distances

Difficulty: ★★ Intermediate

What it Asks

From source S, visit all reachable nodes level by level. Print BFS visit order and distance to each node.

Brain Logic

Use Template 1 (BFS). The order nodes are popped from queue = BFS order.
dist[node] = hops from S. Unreachable nodes have dist = -1.
Sort adj[i] first so BFS explores in ascending neighbor order.

Full Code

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int N, M;
    cin >> N >> M;

    vector<vector<int>> adj(N);
    for (int i = 0; i < M; i++) {
        int u, v; cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    for (int i = 0; i < N; i++) sort(adj[i].begin(), adj[i].end());

    int S; cin >> S;

    vector<int> dist(N, -1);
    vector<int> order;
    queue<int> q;
    dist[S] = 0;
    q.push(S);

    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : adj[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }

    cout << "BFS Order:";
    for (int v : order) cout << " " << v;
    cout << "\nDistances:";
    for (int i = 0; i < N; i++) cout << " " << dist[i];
    cout << "\n";
```

```
    return 0;
}
```

Example

INPUT	OUTPUT
6 5 0 1 0 2 1 3 2 3 3 4 0	BFS Order: 0 1 2 3 4 Distances: 0 1 1 2 3 -1

Q3 DFS & Connected Components

Difficulty: ★★★ Advanced

What it Asks

Run DFS on entire graph (restart from each unvisited node). Count how many connected components (islands) exist.

Brain Logic

Use Template 2 (DFS core loop).
KEY TRICK: every time you start DFS from a NEW unvisited node = 1 new component!
So component count = number of times the outer for-loop triggers DFS.

Full Code

```
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> adj;
vector<bool> visited;
vector<int> dfs_order;

void dfs(int u) {
    visited[u] = true;
    dfs_order.push_back(u);
    for (int v : adj[u])
        if (!visited[v]) dfs(v);
}

int main() {
    int N, M; cin >> N >> M;
    adj.resize(N); visited.assign(N, false);

    for (int i = 0; i < M; i++) {
        int u, v; cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    for (int i = 0; i < N; i++) sort(adj[i].begin(), adj[i].end());

    int components = 0;
    for (int i = 0; i < N; i++) {
        if (!visited[i]) {
            dfs(i);
            components++;   // new island!
        }
    }

    cout << "DFS Order:";
    for (int v : dfs_order) cout << " " << v;
    cout << "\nComponents: " << components << "\n";
    return 0;
}
```

Example

INPUT	OUTPUT
6 3 0 1 1 2 3 4	DFS Order: 0 1 2 3 4 5 Components: 3

Q4 Number of Islands (Grid = Graph)

Difficulty: ★★★ Advanced

What it Asks

Grid of 0s and 1s. Count groups of 1s connected up/down/left/right. Also find the size of the largest group.

Brain Logic

Same as Q3 but on a grid. Each cell (r,c) with value 1 is a node.
Its neighbors: (r-1,c), (r+1,c), (r,c-1), (r,c+1) — if inside grid AND also 1.
Use DFS grid template. Return size from DFS to track largest island.
dr[] dc[] trick: loop d=0..3 instead of writing 4 separate if-statements.

Full Code

```
#include <bits/stdc++.h>
using namespace std;

int R, C;
int grid[100][100];
bool visited[100][100];
int dr[] = {-1, +1, 0, 0};
int dc[] = {0, 0, -1, +1};

int dfs(int r, int c) {
    visited[r][c] = true;
    int size = 1;
    for (int d = 0; d < 4; d++) {
        int nr = r + dr[d], nc = c + dc[d];
        if (nr>=0 && nr<R && nc>=0 && nc<C
            && grid[nr][nc]==1 && !visited[nr][nc])
            size += dfs(nr, nc);
    }
    return size;
}

int main() {
    cin >> R >> C;
    for (int r=0;r<R;r++) for (int c=0;c<C;c++) cin >> grid[r][c];

    int islands = 0, largest = 0;
    for (int r=0;r<R;r++) for (int c=0;c<C;c++) {
        if (grid[r][c]==1 && !visited[r][c]) {
            int sz = dfs(r, c);
            islands++;
            largest = max(largest, sz);
        }
    }
    cout << "Islands: " << islands << "\n";
    cout << "Largest: " << largest << "\n";
    return 0;
}
```

Example

INPUT	OUTPUT
4 5 1 1 0 0 0 1 1 0 0 1 0 0 0 1 1 0 0 0 0 0	Islands: 2 Largest: 4

Q5 Grid Shortest Path (BFS on Grid)

Difficulty: ★★★ Advanced

What it Asks

Grid of 0s (open) and 1s (walls). Find shortest path from top-left (0,0) to bottom-right (R-1,C-1).

Brain Logic

Use BFS grid template (Template 1 grid version). BFS = shortest path guaranteed.
IMPORTANT: if start or end cell is a wall → immediately return -1, no BFS needed.
Move only to cells with value 0 (open). Skip walls (value 1).
dist[R-1][C-1] is your answer after BFS. -1 = unreachable.

Full Code

```
#include <bits/stdc++.h>
using namespace std;

int R, C;
int grid[100][100];
int dist[100][100];
int dr[] = {-1,+1, 0, 0};
int dc[] = { 0, 0,-1,+1};

int bfs() {
    if (grid[0][0]==1 || grid[R-1][C-1]==1) return -1;  // wall check

    for (int r=0;r<R;r++) for (int c=0;c<C;c++) dist[r][c]=-1;

    queue<pair<int,int>> q;
    dist[0][0] = 0;
    q.push({0,0});

    while (!q.empty()) {
        auto [r,c] = q.front(); q.pop();
        for (int d=0;d<4;d++) {
            int nr=r+dr[d], nc=c+dc[d];
            if (nr>=0&&nr<R&&nc>=0&&nc<C&&grid[nr][nc]==0&&dist[nr][nc]==-1) {
                dist[nr][nc] = dist[r][c]+1;
                q.push({nr,nc});
            }
        }
    }
    return dist[R-1][C-1];
}

int main() {
    cin >> R >> C;
    for (int r=0;r<R;r++) for (int c=0;c<C;c++) cin >> grid[r][c];
    int d = bfs();
    cout << "Distance: " << d << "\n";
    cout << "Reachable: " << (d!=-1?"Yes":"No") << "\n";
    return 0;
}
```

Example

INPUT	OUTPUT
3 3 0 0 1 1 0 1 1 0 0	Distance: 4 Reachable: Yes

Q6 Cycle Detection & Bipartite Check (Undirected)

Difficulty: ★★★ Advanced

What it Asks

Two questions in one: (A) Does the undirected graph have a cycle? (B) Is it bipartite (2-colorable)?

Brain Logic

Part A — Cycle in Undirected Graph:
DFS with parent tracking. If we reach a visited node that is NOT our parent → CYCLE.
Why not parent? Because undirected edge A-B appears as A→B and B→A.
B→A is just 'going back' on the same edge, not a cycle. Only flag if it's a DIFFERENT visited node.

Part B — Bipartite:
Try to 2-color the graph with BFS. Color alternates: 0 and 1.
If neighbor has same color as current node → NOT bipartite.
A graph is bipartite ⇔ it has no odd-length cycles.

Full Code

```
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> adj;
vector<bool> visited;
vector<int> color;
bool hasCycle = false, isBipartite = true;

void dfs_cycle(int u, int parent) {
    visited[u] = true;
    for (int v : adj[u]) {
        if (!visited[v]) dfs_cycle(v, u);
        else if (v != parent) hasCycle = true;  // visited, not parent = cycle
    }
}

void bfs_color(int start) {
    queue<int> q;
    color[start] = 0;
    q.push(start);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (color[v]==-1) { color[v]=1-color[u]; q.push(v); }
            else if (color[v]==color[u]) isBipartite = false;
        }
    }
}
```

```
int main() {
    int N, M; cin >> N >> M;
    adj.resize(N); visited.assign(N,false); color.assign(N,-1);
    for (int i=0;i<M;i++) {
        int u,v; cin>>u>>v;
        adj[u].push_back(v); adj[v].push_back(u);
    }
    for (int i=0;i<N;i++) {
        if (!visited[i]) dfs_cycle(i,-1);
        if (color[i]==-1) bfs_color(i);
    }
    cout << "Has Cycle: " << (hasCycle?"Yes":"No") << "\n";
    cout << "Bipartite: " << (isBipartite?"Yes":"No") << "\n";
    return 0;
}
```

Example

INPUT	OUTPUT
4 3 0 1 1 2 2 0	Has Cycle: Yes Bipartite: No

Q7 Reach the Target — Implicit Graph BFS

Difficulty: ★★ Intermediate

What it Asks

From number x, you can do: x+1, x-1, x*2. Stay in [0,LIMIT]. Find fewest moves from S to T.

Brain Logic

Implicit graph = nodes not stored, edges generated on-the-fly by rules.
Each number is a node. The 3 rules are the edges.
BFS still works! dist[x] = min moves to reach x from S.
Generate neighbors: x+1, x-1, x*2. Only valid if in [0,LIMIT] and not visited.

Full Code

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int LIMIT, S, T;
    cin >> LIMIT >> S >> T;

    vector<int> dist(LIMIT+1, -1);
    queue<int> q;
    dist[S] = 0;
    q.push(S);

    while (!q.empty()) {
        int x = q.front(); q.pop();
        // Generate neighbors by the 3 rules
        for (int nx : {x+1, x-1, x*2}) {
            if (nx>=0 && nx<=LIMIT && dist[nx]==-1) {
                dist[nx] = dist[x]+1;
                q.push(nx);
            }
        }
    }

    cout << "Steps: " << dist[T] << "\n";
    cout << "Reachable: " << (dist[T]!=-1?"Yes":"No") << "\n";
    return 0;
}
```

Example

INPUT	OUTPUT
10 2 9	Steps: 3 Reachable: Yes

Trace: 2 → 4 (×2) → 8 (×2) → 9 (+1). Three moves!

Q8 Knight Moves — BFS on Chess Board

Difficulty: ★★★ Advanced

What it Asks

Chess knight on N×N board. 8 L-shaped moves. Find min moves from (sr,sc) to (tr,tc).

Brain Logic

Same as BFS grid template BUT neighbors are the 8 knight moves, not 4 directions.
State = (row, col) pair. dist[r][c] = min moves to reach (r,c).
Store all 8 move offsets in arrays dr[] dc[] (like grid BFS but 8 entries).

Full Code

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int N, sr, sc, tr, tc;
    cin >> N >> sr >> sc >> tr >> tc;

    // All 8 knight L-moves
    int dr[] = {-2,-2,-1,-1,+2,+2,+1,+1};
    int dc[] = {-1,+1,-2,+2,-1,+1,-2,+2};

    vector<vector<int>> dist(N, vector<int>(N,-1));
    queue<pair<int,int>> q;
    dist[sr][sc] = 0;
    q.push({sr,sc});

    while (!q.empty()) {
        auto [r,c] = q.front(); q.pop();
        for (int i=0;i<8;i++) {
            int nr=r+dr[i], nc=c+dc[i];
            if (nr>=0&&nr<N&&nc>=0&&nc<N&&dist[nr][nc]==-1) {
                dist[nr][nc] = dist[r][c]+1;
                q.push({nr,nc});
            }
        }
    }
    cout << "Moves: " << dist[tr][tc] << "\n";
    return 0;
}
```

Example

INPUT	OUTPUT
8 0 0 7 7	Moves: 6

Q9 Directed Reachability — BFS on Directed Graph

Difficulty: ★★ Intermediate

What it Asks

In a directed graph, from source S, find all reachable vertices. Print BFS order and count.

Brain Logic

Same BFS as Q2, but directed: adj[u].push_back(v) only (no reverse edge).
Sort adj[i] so BFS visits neighbors in ascending order.
Count = size of BFS order list (includes source itself).

Full Code

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int N, M; cin >> N >> M;
    vector<vector<int>> adj(N);
    for (int i=0;i<M;i++) {
        int u,v; cin>>u>>v;
        adj[u].push_back(v);   // directed: only one way
    }
    for (int i=0;i<N;i++) sort(adj[i].begin(),adj[i].end());

    int S; cin >> S;
    vector<bool> visited(N,false);
    vector<int> order;
    queue<int> q;
    q.push(S); visited[S]=true;

    while (!q.empty()) {
        int u=q.front(); q.pop();
        order.push_back(u);
        for (int v:adj[u]) if(!visited[v]) { visited[v]=true; q.push(v); }
    }

    cout << "BFS Order:";
    for (int v:order) cout<<" "<<v;
    cout << "\nReachable: " << order.size() << "\n";
    return 0;
}
```

Example

INPUT	OUTPUT
6 5	BFS Order: 0 1 2 3 4

0 1 0 2 1 3 2 3 3 4 0	Reachable: 5

Q10 Directed Cycle Detection & Sources

Difficulty: ★★★ Advanced

What it Asks

Detect cycle in directed graph. Count sources (vertices with in-degree 0 = no incoming edges).

Brain Logic

Use DFS 3-color template (Template 2, 3-color version).
WHITE=0, GRAY=1 (in current path), BLACK=2 (fully done).
Reach a GRAY node during DFS → CYCLE found.
Reach a BLACK node → safe, already fully explored, no new cycle.
Sources: count nodes where in_degree[i] == 0 after reading all edges.

Full Code

```
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> adj;
vector<int> color;   // 0=white 1=gray 2=black
bool has_cycle = false;

void dfs(int u) {
    color[u] = 1;
    for (int v : adj[u]) {
        if (color[v]==1) has_cycle=true;   // gray = cycle!
        if (color[v]==0) dfs(v);
    }
    color[u] = 2;
}

int main() {
    int N, M; cin >> N >> M;
    adj.resize(N); color.assign(N,0);
    vector<int> in_deg(N,0);
    for (int i=0;i<M;i++) {
        int u,v; cin>>u>>v;
        adj[u].push_back(v);
        in_deg[v]++;
    }
    for (int i=0;i<N;i++) if(color[i]==0) dfs(i);

    int sources=0;
    for (int i=0;i<N;i++) if(in_deg[i]==0) sources++;

    cout << "Has Cycle: " << (has_cycle?"Yes":"No") << "\n";
    cout << "Sources: " << sources << "\n";
    return 0;
}
```

Example

INPUT	OUTPUT
3 3 0 1 1 2 2 0	Has Cycle: Yes Sources: 0

Q11 Topological Sort — Kahn's Algorithm

Difficulty: ★★★ Advanced

What it Asks

Order vertices of a DAG so every vertex comes AFTER all its prerequisites. Pick lexicographically smallest order. Report CYCLE if none exists.

Brain Logic

Use Template 3 (Kahn's). Steps:

1. Find all nodes with in_degree=0 → push to min-heap.
2. Pop smallest, add to result, reduce in_degree of all its neighbors.
3. If neighbor's in_degree hits 0 → push to heap.
4. If result size < N at end → CYCLE (some nodes were stuck waiting).

Min-heap = priority_queue<int, vector<int>, greater<int>> ← smallest first!

Full Code

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int N, M; cin >> N >> M;
    vector<vector<int>> adj(N);
    vector<int> in_deg(N,0);
    for (int i=0;i<M;i++) {
        int u,v; cin>>u>>v;
        adj[u].push_back(v);
        in_deg[v]++;
    }

    priority_queue<int,vector<int>,greater<int>> pq;
    for (int i=0;i<N;i++) if(in_deg[i]==0) pq.push(i);

    vector<int> order;
    while (!pq.empty()) {
        int u=pq.top(); pq.pop();
        order.push_back(u);
        for (int v:adj[u]) { in_deg[v]--; if(in_deg[v]==0) pq.push(v); }
    }

    if ((int)order.size()<N) {
        cout << "Order: CYCLE\nAcyclic: No\n";
    } else {
        cout << "Order:";
        for (int v:order) cout<<" "<<v;
        cout << "\nAcyclic: Yes\n";
    }
    return 0;
}
```

Example

INPUT	OUTPUT
6 6 5 2 5 0 4 0 4 1 2 3 3 1	Order: 4 5 0 2 3 1 Acyclic: Yes

Q12 Build Levels — Longest Path in DAG

Difficulty: ★★★ Advanced

What it Asks

In a DAG, find the longest dependency chain counted in nodes (a single node = 1 stage). Also count sinks (nodes with out-degree 0).

Brain Logic

Run Kahn's topo sort. While processing, update stage[] with DP:
stage[v] = max(stage[v], stage[u] + 1) for each edge u→v
This works because in topo order, when we process u, all nodes leading to u are done.
So stage[u] is final. We can safely compute stage[v].
Answer = max value in stage[]. Single node with no edges → stage=1.
Sinks = nodes where out_degree == 0.

Full Code

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int N, M; cin >> N >> M;
    vector<vector<int>> adj(N);
    vector<int> in_deg(N,0), out_deg(N,0);

    for (int i=0;i<M;i++) {
        int u,v; cin>>u>>v;
        adj[u].push_back(v);
        in_deg[v]++;
        out_deg[u]++;
    }

    priority_queue<int,vector<int>,greater<int>> pq;
    for (int i=0;i<N;i++) if(in_deg[i]==0) pq.push(i);

    vector<int> stage(N,1);   // every node starts at stage 1
    int max_stage=1;

    while (!pq.empty()) {
        int u=pq.top(); pq.pop();
        for (int v:adj[u]) {
            stage[v] = max(stage[v], stage[u]+1);  // DP update
            max_stage = max(max_stage, stage[v]);
            in_deg[v]--;
            if(in_deg[v]==0) pq.push(v);
        }
    }

    int sinks=0;
    for (int i=0;i<N;i++) if(out_deg[i]==0) sinks++;
```

```
    cout << "Stages: " << max_stage << "\n";
    cout << "Sinks: " << sinks << "\n";
    return 0;
}
```

Example

INPUT	OUTPUT
4 4 0 1 1 2 2 3 0 3	Stages: 4 Sinks: 1

Trace: stage[0]=1 → stage[1]=2 → stage[2]=3 → stage[3]=max(2,4)=4. Max=4. Only node 3 has out_degree=0.

MASTER CHEAT SHEET

Q	Topic	Template to Use	Key Trick	Output
1	Graph Stats	Read graph	adj[i].size() = degree	Vertices, Edges, Degrees, MaxDeg
2	BFS Traversal	Template 1 BFS	Sort adj[] first	BFS Order, Distances
3	DFS Components	Template 2 DFS	Each outer DFS call = +1 component	DFS Order, Components
4	Islands (Grid)	DFS Grid	dr[]/dc[] trick, return size	Islands, Largest
5	Grid Shortest Path	BFS Grid	Check start/end not wall first	Distance, Reachable
6	Cycle + Bipartite	DFS+parent & BFS 2-color	v!=parent check; color=1-color[u]	Has Cycle, Bipartite
7	Implicit BFS	Template 1 BFS	Generate nexts inline: {x+1,x-1,x*2}	Steps, Reachable
8	Knight Moves	BFS Grid (8 dirs)	8-entry dr[]/dc[] arrays	Moves
9	Directed BFS	Template 1 BFS	One-way adj, sort, count order	BFS Order, Reachable
10	Directed Cycle	Template 2 DFS 3-color	GRAY=cycle; count in_deg==0	Has Cycle, Sources
11	Topo Sort	Template 3 Kahn's	Min-heap; if count<N → CYCLE	Order or CYCLE, Acyclic
12	Build Levels	Kahn's + DP	stage[v]=max(stage[v],stage[u]+1)	Stages, Sinks

Top 5 Bugs That Kill Marks

⚠ AVOID THESE

1. Undirected graph: forgot adj[v].push_back(u). Only added one direction. → ALL results wrong.
2. BFS: marked visited AFTER pop (not before push). Same node added to queue multiple times.
3. Cycle in directed graph: used 2-state visited instead of 3-color. Flags back-edges as cycles.
4. Topo sort: used regular queue instead of min-heap. Got valid topo order but not lex-smallest.
5. Grid BFS: forgot to check if start/end is a wall before running BFS.

Quick Copy-Paste Snippets

```
// Read undirected graph
vector<vector<int>> adj(N);
for(int i=0;i<M;i++){int u,v;cin>>u>>v;adj[u].push_back(v);adj[v].push_back(u);}
```

```
// BFS from S
vector<int> d(N,-1);queue<int>q;d[S]=0;q.push(S);
while(!q.empty()){int u=q.front();q.pop();for(int v:adj[u])if(d[v]==-
1){d[v]=d[u]+1;q.push(v);}}

// DFS component count
int comp=0;for(int i=0;i<N;i++)if(!vis[i]){dfs(i);comp++;}

// Kahn's topo sort
priority_queue<int,vector<int>,greater<int>>pq;
for(int i=0;i<N;i++)if(in[i]==0)pq.push(i);
while(!pq.empty()){int u=pq.top();pq.pop();res.push_back(u);for(int v:adj[u])if(--
in[v]==0)pq.push(v);}

// Grid 4-dir check
int dr[]={-1,1,0,0};int dc[]={0,0,-1,1};
for(int d=0;d<4;d++){int nr=r+dr[d],nc=c+dc[d];if(nr>=0&&nr<R&&nc>=0&&nc<C)...}
```